

**FOLLOW-ON WORK**

# Development Environment & Scale Validation

PULSE has no development environment. This document states the gap plainly, sets out how to stand up a development SharePoint site, how to load it heavily enough to find the breaking points, and how to evaluate Dataverse as an alternative store.

**IMPORTANT**

This is planned work, not a description of something that exists. Nothing in this document should be read as a capability PULSE has today.

| STATUS          | OWNER      | BLOCKS                 | OUTPUT                    |
|-----------------|------------|------------------------|---------------------------|
| Not yet started | Unassigned | Confident scale claims | A written decision record |

## THE GAP

## 01 / THERE IS NO DEVELOPMENT ENVIRONMENT

PULSE is currently developed against one of two things: browser-local fallback state, which is not backed by SharePoint at all, or a real SharePoint site holding real records. There is no third option — no isolated site whose data can be destroyed and rebuilt freely, and no dataset large enough to show where the application stops behaving well.

**WHAT THIS MEANS TODAY**

| AREA                 | CURRENT SITUATION                                                                  |
|----------------------|------------------------------------------------------------------------------------|
| Isolated environment | None. No development or staging site is defined in configuration or documentation. |
| Automated tests      | None. There is no test directory and no test script.                               |
| Schema provisioning  | Exercised only against sites that already matter.                                  |
| Scale evidence       | None. Behaviour past a few hundred records per list is unmeasured.                 |
| Rollback rehearsal   | Documented as a procedure but never practised against a disposable site.           |

**CAUTION**

Because provisioning and destructive operations have only ever run against sites in use, a schema mistake currently has no safe place to be discovered. That is the most immediate reason to do this work, ahead of any performance question.

Local fallback mode is useful for interface work and is what the screenshot captures use. It cannot substitute for a SharePoint site, because it exercises none of the REST transport, provisioning, permission, paging, or identity behaviour that carries the real risk.

## STEP ONE

## 02 / STAND UP A DEVELOPMENT SITE

The goal is a site nobody depends on, holding nothing real, that can be provisioned from empty and thrown away. Treat reaching that state as the first deliverable, before any load testing.

1. Request a separate SharePoint site collection for PULSE development, with the requesting team as owners. Confirm it is outside any retention or records scope that would make deletion difficult.
2. Publish the current PULSE package to a page on that site.
3. Run SharePoint Setup and let it provision the schema into an empty site. Record every list and column it creates, and every failure, because this is the first time that path is observed from a clean start.
4. Confirm identity and role resolution behave as expected for at least one administrator and one ordinary member.
5. Seed synthetic records only. Use neutral role display names and example.mil addresses, matching the convention already used for documentation captures.
6. Write down how to rebuild the site from empty, so it can be reset between experiments rather than repaired.

**NOTE**

The application already supports pointing at an explicit site through the `manualSharePointSiteUrl` value in `app-config.js`, which is empty today. That is the intended hook for aiming a local build at a development site.

Do not copy production records into the development site. Generate synthetic data instead; the volumes needed for scale testing are far larger than anything worth copying, and copying imports real personal and project information into a site with weaker controls.

## KNOWN LIMITS

## 03 / SCALE CEILINGS ALREADY IN THE CODE

These are not hypotheses. They were read from the current source and are the specific things a large dataset would exercise first.

### PAGING AND THE REQUEST GUARD

The REST adapter's item fetch follows SharePoint's continuation link in a loop bounded by a guard of 50 requests. When the guard is reached the loop simply stops and returns what it has. There is no error, no warning, and no marker on the result.

| LIST LOAD STYLE           | PAGE SIZE       | CEILING BEFORE SILENT TRUNCATION |
|---------------------------|-----------------|----------------------------------|
| No explicit page size     | 100 per request | about 5,000 items                |
| Explicit page size of 500 | 500 per request | about 25,000 items               |

**IMPORTANT**

Silent truncation is the most dangerous behaviour listed in this document. Past the guard, the application shows a complete-looking view of an incomplete dataset. A scale test must check record counts against the list, not just confirm the page renders.

### THE LIST VIEW THRESHOLD

Three loads sort server-side on a date column: issues and tickets on their occurrence date, and the audit log on its action time. These are exactly the lists that grow without bound. Once a list passes SharePoint's 5,000-item view threshold, a sorted or filtered query against an unindexed column fails rather than degrading. Adding the right indexed columns is likely to be a prerequisite, not an optimisation.

## KNOWN LIMITS

## 04 / THE REFRESH AND MEMORY MODEL

PULSE holds the whole workspace in memory. Boot reads every PULSE list, plus a field-definition probe per list, and assembles a single object graph. Background refresh does not fetch deltas; it repeats that load and replaces the graph wholesale.

**HOW OFTEN THAT HAPPENS**

- On a timer, gated so a successful refresh runs no more often than every five seconds, and only while the tab is visible.
- On navigation between areas.
- On the tab regaining focus.

At small data volumes this is invisible and gives the application its live feel. The open question is what it costs once each list holds thousands of items: the request count multiplies, the parse and mapping work repeats, and each refresh allocates a fresh copy of the entire workspace.

**NOTE**

Whether the answer is delta loading, a longer interval, refreshing only the active area, or a different store is exactly what this evaluation should decide. It should not be decided from intuition, which is the current situation.

## STEP TWO

## 05 / BUILD A LARGE DATA SET

Load the development site past the ceilings above, not up to comfortable numbers. The point is to find the failure, so the volumes should be deliberately unreasonable and then reduced until behaviour is acceptable.

| RECORD TYPE           | SUGGESTED VOLUME       | WHY THIS NUMBER                                           |
|-----------------------|------------------------|-----------------------------------------------------------|
| Projects              | 500 and 2,000          | Drives dashboard, overview, and every project roll-up     |
| Action items          | 50,000 across projects | Crosses both paging ceilings and the view threshold       |
| Risks                 | 5,000                  | Loaded in full and grouped per project on every load      |
| Travel requests       | 10,000                 | Feeds My Travel filtering and the calendar                |
| Audit log             | 100,000                | Sorted server-side; passes the threshold by a wide margin |
| Issues and tickets    | 20,000                 | Sorted server-side and paged at 500                       |
| Document review items | 5,000                  | Bucketed into review columns during load                  |

Generate the data with a script against the development site's REST endpoints so volumes are reproducible and the site can be rebuilt to a known state. Keep the generator in the repository alongside the capture script, so a later maintainer can recreate the conditions rather than trust a past result.

**CAUTION**

Create this data only on the development site. A generator pointed at the wrong site by an empty or stale configuration value is a realistic accident; require the target site to be passed explicitly rather than defaulted.

## STEP THREE

## 06 / WHAT TO MEASURE AND PROVOKE

### MEASURE

- Time from page load to a usable interface, and the number of REST requests it took.
- Time for one background refresh, and whether refreshes begin to overlap or queue.
- Time to render the heaviest views: the tracker for a large project, the overview roll-up, the travel calendar, and the audit log.
- Memory held after boot and after an hour of idling with refreshes running.
- Time for a single save to complete under load, since saves compete with refreshes.

### PROVOKE

1. Truncation: compare counts shown in the application against counts in the list, on every list, past the guard.
2. Threshold failures: sort and filter the audit log and issue lists past 5,000 items with and without indexed columns.
3. Render stalls: open the largest project's tracker and confirm whether the interface remains responsive.
4. Save behaviour under load: edit records rapidly while refreshes run, and confirm nothing is lost or reverted.
5. Reload the page repeatedly under load, which is how a user reacts to slowness and doubles the request pressure.

### FINAL CHECK

Record the numbers, the site state, and the package version with every result. A performance claim without those three is not reproducible and will not survive the next change.

STEP FOUR

07 / EVALUATING DATAVERSE

Dataverse is worth evaluating as a store because the pressures above are the ones SharePoint Lists handle least well: server-side querying at volume, relational integrity between records, and predictable behaviour past the view threshold. Nothing in the current source references Dataverse; this would be new work with no existing commitment.

WHAT TO COMPARE, USING THE SAME DATA SET

| DIMENSION       | QUESTION TO ANSWER                                                                              |
|-----------------|-------------------------------------------------------------------------------------------------|
| Query at volume | Does filtering and sorting stay predictable past the volumes where lists fail?                  |
| Paging          | Can a page of data be fetched without the application holding the whole workspace?              |
| Relationships   | Do project, task, and risk relationships become enforceable rather than mapped by code?         |
| Identity        | Does the existing page session still supply identity, or is a new authentication path required? |
| Licensing       | What does each user cost, and who approves that?                                                |
| Migration       | Can existing records move without loss, and can it be reversed?                                 |
| Hosting         | Does the application still ship as a single-file package on a page?                             |

IMPORTANT

Evaluate against the loaded development site, not a small proof of concept. A store change that is not measured under the volumes that motivated it proves nothing.

Note the architectural consequence honestly: PULSE currently has no server, no database, and no separate sign-in, and its simplicity follows from that. Introducing Dataverse trades that simplicity for capability, and the evaluation should state the cost as clearly as the benefit.

## OUTPUT

## 08 / WHAT A FINISHED EVALUATION PRODUCES

The work is complete when the following exist and are recorded, not when the site has been built.

1. A development site that can be provisioned from empty and rebuilt to a known state, with the procedure written down.
2. A data generator held in the repository, with the volumes it produces.
3. Measured results for boot, refresh, render, memory, and save under load, each stamped with package version and site state.
4. A list of confirmed failure points, with the volume at which each appears.
5. A written recommendation on the refresh and paging model, with the evidence behind it.
6. A written recommendation on Dataverse: adopt, reject, or revisit at a stated threshold, with the comparison behind it.

**OPERATING STANDARD**

Until these results exist, PULSE should make no claim about the data volumes it supports. Stating a limit that has never been measured is worse than stating that the limit is unknown.

**RELATED MATERIAL**

- Standard Operating Procedures, SOP 10, for the controlled form of this procedure.
- Technical Reference, for the data model, refresh behaviour, and packaging this document measures.
- Document 09, Focused Tools & Package Delivery, since every focused package shares this data layer and inherits these ceilings.